

Blog Posting Pipeline: Demo

Brief

Explainer for `blog-posting-pipeline-demo`

The short version

What this is

A static web page that *acts out* a private production system rather than being one. The real `blog-posting-pipeline` writes and publishes marketing articles for paying clients; its source stays private because the pipeline is the agency’s product. This repository is its public stand-in: three hand-written example runs for an invented coffee brand, replayed on a timer, so a reader can see the real architecture’s shape without the real system being exposed. Nothing it “generates” was generated. No model is called, no request is made. Every run number was typed into a JavaScript file by hand.

One thing on the page *is* real, and that is the point: the Blog Library holds fifteen genuine articles the real pipeline actually published for a real client. They are the only evidence on the page. The runs beside them are not evidence, and are labelled as such.

1 Why it exists

“Trust me, I built a seven-stage AI pipeline” is not evidence, and the source cannot be shown. So the project asks a narrower question: can the *architecture* be demonstrated honestly, in a form someone can click, while the implementation stays private?

The trade it makes: give up all of the real system’s substance (prompts, client rules, code) and keep only its shape (seven stages, in order, with the right kind of data moving between them). Then be relentless about labelling, because a convincing demo is exactly the thing most likely to be mistaken for the real product.

2 The provenance ladder

The central design idea. Three kinds of content share one page, and confusing them would be the failure. Each tier gets its own disclosure mechanism.

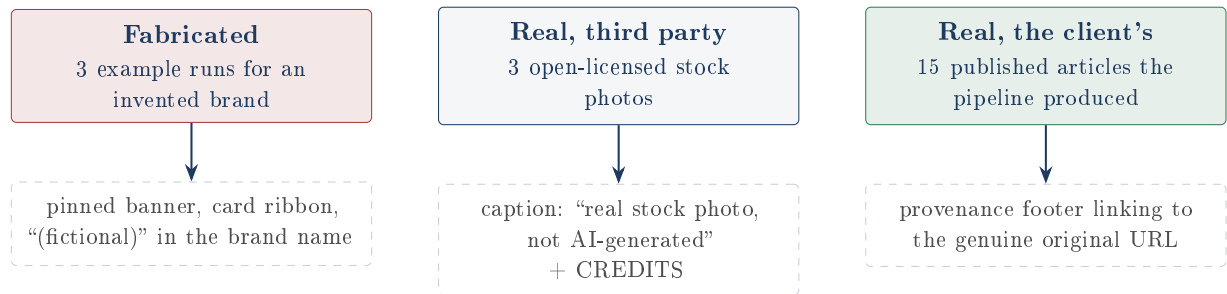


Figure 1: Three origins, three disclosure mechanisms.

The tempting design is to invent a fake blog library too, so everything is uniformly fictional and no disclosure question arises. The project rejects that: the client’s posts are already public, the real pipeline genuinely produced them, so showing them (with a link back to each original) is

the only actual proof on the page that the system ships anything. The cost of that choice is that real and fake now sit in one grid, and the labelling carries the whole burden.

3 Architecture at a glance

No server, no framework, no build step, no dependencies. About 1,700 lines of hand-written code across four files.

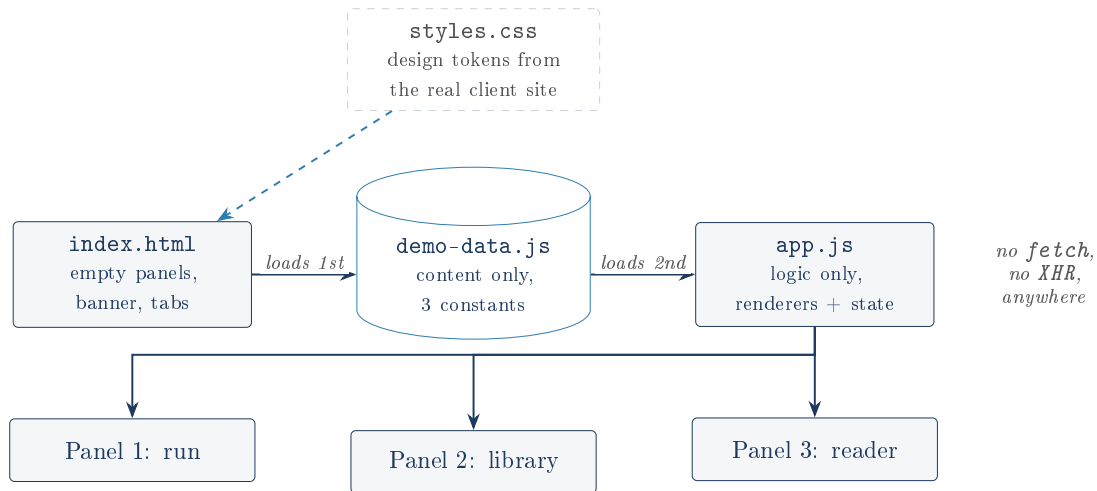


Figure 2: The clean split: `demo-data.js` is content with no behaviour, `app.js` is behaviour with no content.

The data file holds three constants: `REAL_DAVONEX_POSTS` (15 real posts, full article bodies, roughly 154,000 characters, which is most of the file’s 231 KB), `DEMO_RUNS` (3 fabricated runs of 7 steps each), and `PRESETS` (the chip labels, which also link each fabricated run to the real post whose shape inspired it).

4 The seven stages

The rail across the top is the demo’s central claim: *this is the real system’s architecture*.

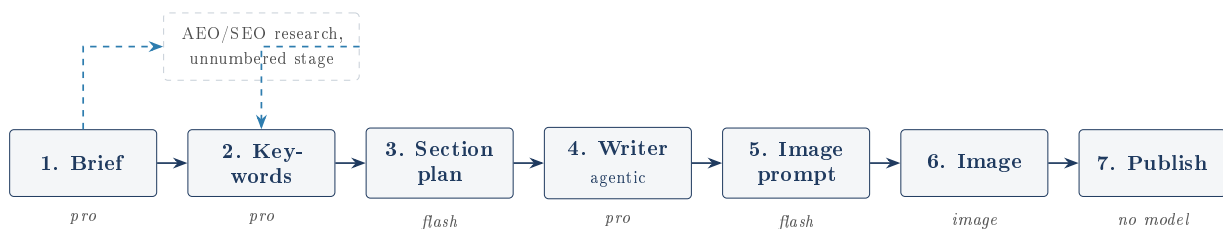


Figure 3: The seven stages. The unnumbered research stage is not on the visible rail but does appear as a line in the cost table, which is a creditable piece of fidelity.

Architectural fidelity: verified, and it holds

The demo’s stages were compared against the real private project’s own documentation. The stage names, their order, the tiering of expensive against cheap models, the specific model identifiers, the fact that stage 4 is an agentic loop with exactly two tools, the unnumbered research stage, and the publish stage’s trio of actions all match.

This is the demo’s strongest result: its architectural claim is accurate, not marketing.

5 How a “run” actually works

Clicking “Run the pipeline” computes nothing. All seven steps are already in the page before the click; the button reveals one more of them every 900 milliseconds by adding a CSS class, then adds a fabricated card to the library with a fifteen-minute countdown.

Two consequences worth stating out loud:

- **The run cannot fail**, because nothing runs. The real system’s failure modes (a refusal, a timeout, a malformed reply, a rejected publish) have no representation here at all.
- **The finished article is in the page from the start**. Anyone who opens developer tools before clicking sees the whole post. That is the literal meaning of “scripted.”

6 What was verified, by running it

The page was loaded in a real headless browser, driven through a full run, and instrumented. The claims below were tested, not trusted.

Claim	Result	Evidence
“No live API calls happen here”	True	Zero network requests to any non-local origin, across the entire flow.
Self-contained (fonts, images local)	True	The only <code>url()</code> in the CSS is a bundled font. No CDN.
Free of JavaScript errors	True	Zero page and console errors.
Seven stages step through	True	0 of 7 visible before the click, 7 of 7 after.
“All 15 real posts”	True	15 entries, 15 image files, no broken paths, no orphans.
The countdown and expiry work	True	Counter went (15) to (16); ribbon ticked 14:58 to 14:56.
Cost range is the real published figure	Partly false	Only 1 of 3 presets matches. See below.

The headline result

The demo’s central promise, that it is inert and self-contained, is true and provably so. Driven through its whole flow in a real browser, the page issued **no network requests whatsoever**.

7 The findings that matter

The cost caption claims fabricated numbers are real

Under every run, the page prints: “The total range is the real, published per-post cost for the production system (see its README).” That is a claim of fact, and it is true for only one of the three presets. The demo gives each preset a different total range; checked against the real project’s own published figure, the first matches exactly and the other two do not. They overshoot it at both ends. They are fabricated variations displayed under a caption asserting

they are real.

The demo's README compounds this by describing its headline figure as “anchored to” the source project's published range. The number it quotes is the spread across the demo's own three presets, not the source's published range. It reads as a citation and is not one.

This is the one place where a project built entirely around honest labelling labels fabricated content as real. It is fixable in minutes: set all three presets to the real published range, or say the totals are illustrative too. The first is better. Fix it before showing the page to anyone.

The library grid is rebuilt from scratch every second, for 15 minutes

The countdown ribbon ticks via `setInterval(renderLibrary, 1000)`, but `renderLibrary` does not update the ribbon: it reassigns the whole grid's `innerHTML`, destroying and recreating all 16 cards and all 16 images, roughly 900 times over a card's lifetime, to change five characters. Measured, not inferred: a mutation observer recorded 4 grid rebuilds in 4 seconds with a demo card live, and 0 without one. It is observable, too. An attempt to click a card through browser automation failed with “Node is detached from document,” because the card was destroyed between being found and being clicked. Real users survive it only because the click handler is bound to the container rather than the cards; what does not survive is text selection, which is wiped once a second. The timer also keeps rebuilding the grid while it is hidden behind the reader panel.

The articles claim citations they do not contain

The README describes the Writer step as producing “numbered citations with a References list,” matched deliberately to the real system's prompt templates. The reference lists are real and numbered; the inline citations are absent. Searching all three fabricated articles for a marker of the form [1] returns zero matches. The prose cites nothing and a numbered source list simply appears at the end. Since the entire editorial argument for this kind of content is that claims are attributable, this is a gap in the demo's most load-bearing artifact.

Smaller, in descending order of interest:

- **One preset's arithmetic does not close.** Its per-step line items sum to \$0.422, above the \$0.42 upper bound of the total printed directly below them.
- **No tab looks selected while reading a post.** The reader is a third panel with no tab button, so `switchTab("reader")` matches nothing and deactivates both tabs. Confirmed in the browser: zero tabs marked active while reading.
- **escapeHtml does not escape quotes** yet its output is dropped into HTML attributes. Not exploitable today, since every string is a hand-written constant and no user input reaches a renderer, but it is a guard that looks complete and is not.
- **The same sources render two ways:** step 4 prints explicit [1], [2] markers from a `ref` field, while the step 7 preview drops `ref` and lets the list auto-number. They agree today only by luck of ordering.
- **All six fabricated sources point at example.com.** The honest choice, but the References lists lead nowhere.
- **Post cards are not keyboard accessible:** plain `<article>` elements with no `tabindex`, opened by a delegated click.
- **The page is noindex,** which is correct: it reproduces a client's articles verbatim and must not compete with the client's own pages for the rankings the real pipeline exists to win.

8 Licensing: handled unusually well

Three licences apply to one folder. The repository’s own code is PolyForm Strict. Two stock photos are public domain and CC BY. The third is CC BY-SA 4.0, and `CREDITS.md` explicitly records that this one cropped file stays CC BY-SA and does *not* fall under the repository’s licence.

The share-alike catch, spotted

CC BY-SA is viral: a derivative must carry the same licence, and a cropped photo is a derivative. Dropping it into a strictly licensed repository without comment would quietly violate its terms. Most portfolio projects never notice. This one carved it out by name, and can say so.

The fifteen real posts are not open-licensed at all; they are the client’s. They are reproduced with a provenance link on every article and a `CREDITS.md` explaining the reasoning. The demo also refuses to hotlink: everything is fetched once and stored locally, which is what makes the “no live calls” guarantee airtight rather than nearly airtight.

9 What it cannot demonstrate

Does show

- The real stage list, order, model tiering (independently verified)
- The shape of the data and the output
- That the real system ships: 15 real posts
- Front-end craft and disclosure design
- Judgment about what may be shown



Cannot show

- Any prompt or prompt engineering
- That the pipeline works (nothing runs)
- Output quality (hand-written to look good)
- Failure handling (no error path exists)
- Latency (900 ms is a UI pause)
- Cost measurement (typed in; 2 are wrong)
- Any backend skill at all

Figure 4: The boundary of the evidence.

How to talk about it

The worst possible framing is “here is my AI pipeline running.” It is not running, one question exposes that, and every other claim then becomes suspect.

The strong framing is: “the real system is private, so I built an honest replica of its architecture and put it beside fifteen posts the real system actually published. The runs are fabricated and labelled as such. Here is what it proves, and here is what it cannot.” That turns the demo’s greatest weakness, that it is fake, into evidence of the judgment that produced it. Fix the cost caption first: it is the one place the page’s own discipline slips, and it slips in the exact direction an interviewer will probe.

Glossary

Static site

A site made only of files handed over unchanged, with no program running on a server. Everything happens in your browser. This is why the demo can make no live calls: there is nothing to call.

Vanilla JavaScript

JavaScript with no framework. The programmer edits the page directly. Removes a whole toolchain; costs you the free optimizations a framework gives, one of which is missed here.

DOM

The browser's live, tree-shaped model of the page. JavaScript changes what you see by editing this tree.

innerHTML

Assign to it and the browser throws away an element's contents and rebuilds them from scratch. Quick to write; it destroys rather than updates, which is the root of the once-a-second rebuild defect.

JSON

A plain-text format for structured data. How the real system's stages hand results to each other; showing it raw signals "a machine read this, not a human."

SEO / AEO

SEO shapes a page so a search engine ranks it and a human clicks. AEO shapes a page so an AI assistant answering the question directly cites you as its source. With AEO there may be no click at all, so the goal shifts from ranking to being quotable.

Agentic loop

The model is given tools and loops (think, call a tool, read the result, think again) until it decides it is done, rather than answering once. Stage 4 does this with two tools, which is why it dominates the cost table.

Embedding / Pinecone

An embedding turns text into a list of numbers placed so similar meanings sit near each other; Pinecone searches those by nearness, not keyword. The real system uses it to avoid repeating itself and to find internal links.

IndexNow

A protocol for telling search engines "this URL just changed, come look now." One request. Shown here as `not sent (demo)`.

Slug

The readable identifier in a web address. Here it doubles as each post's key.

Design tokens

Named values for a design's raw ingredients (a colour, a corner radius), defined once and referenced everywhere so the design stays consistent.

Escaping

Rewriting characters like `<` into harmless stand-ins so text is displayed rather than executed. Skipping it causes cross-site scripting bugs.

PolyForm Strict

A licence granting the right to read and evaluate, reserving essentially everything else. Standard for showing work without licensing its reuse.

CC BY-SA

An open licence with a share-alike term: derivatives must carry the same licence. Viral, and easy to violate by accident.

*Every finding here was taken from the source, or from driving the page in a real browser. Claims about the private **blog-posting-pipeline** were checked against that project's own documentation; none of its private detail is reproduced here.*